

Task Management System

Software Engineering Documentation | MERN Stack | Full-Stack Project

TECHNOLOGY STACK	MongoDB · Express.js · React.js · Node.js
TARGET USERS	Managers and Employees
DEPLOYMENT	Docker · Vercel · Heroku · MongoDB Atlas
ARCHITECTURE	RESTful API Backend + React SPA Frontend
AUTHENTICATION	JWT-based + Google OAuth
VERSION	1.0.0 Production Ready
COURSE	Software Engineering

Task Management is a production-ready, collaborative task management SaaS platform engineered on the MERN stack. It enables organizations to plan, assign, track, and close tasks through a role-based web interface — eliminating scattered spreadsheets and fragmented communication channels.

Table of Contents

1. Project Overview

- 1.1 Application Summary
- 1.2 Project Goals & Scope
- 1.3 Stakeholders
- 1.4 Problem Statement

2. Software Requirements Specification (SRS)

- 2.1 Functional Requirements
- 2.2 Non-Functional Requirements
- 2.3 Use Cases & User Stories
- 2.4 System Constraints

3. System Design

- 3.1 Architectural Design
- 3.2 Database Design
- 3.3 System Components
- 3.4 UML Diagrams

4. Implementation

- 4.1 Development Plan
- 4.2 Coding Best Practices
- 4.3 Testing & Debugging

5. Deployment & Evolution

- 5.1 Deployment Strategy
- 5.2 System Monitoring
- 5.3 Maintenance & Evolution

1

Project Overview

1.1 Application Summary

Project Name	TaskFlow — Task Management SaaS Platform
Technology Stack	MongoDB · Express.js · React.js · Node.js (MERN)
Target Users	Managers and Employees (Team Members)
Deployment Target	Docker · Heroku · Vercel · MongoDB Atlas
Architecture	RESTful API Backend + React SPA Frontend
Authentication	JWT-based with Google OAuth support
Version	1.0.0 Production Ready
Course	Software Engineering

1.2 Project Goals & Scope

TaskFlow addresses the persistent challenge modern teams face when coordinating work across multiple contributors without a unified platform. The application provides a centralized, role-aware system that replaces scattered spreadsheets, email threads, and fragmented communication.

Target Users:

- **Managers** — Full control over projects, task assignment, team analytics, and member management.
- **Employees** — Focused view of assigned tasks, personal analytics, and collaboration features.

Key Features & Functionalities:

- Secure JWT authentication with Google OAuth social login
- Role-Based Access Control (RBAC) distinguishing Manager and Employee roles
- Full task lifecycle management: creation, assignment, prioritization, status tracking, and deletion
- Project management — managers can create, organize, and archive projects
- Advanced task filtering: by status, priority, project, assignee, date range, and overdue flag
- Real-time analytics dashboards with personal and team performance metrics
- Collaborative task comments for team communication
- Private task creation for personal to-dos
- Overdue task detection and reassignment capability for managers
- Responsive UI built with TailwindCSS — desktop and tablet optimized

Constraints:

- **Technical:** Requires Node.js 14+, MongoDB 4.4+; JWT tokens stored in localStorage (stateless)
- **Business:** Two roles only (Manager / Employee); no granular permission scopes in v1.0
- **Legal:** Passwords hashed with bcryptjs; Google OAuth compliant with OAuth 2.0 standard
- **Scalability:** Horizontal scaling via Docker; MongoDB Atlas supports sharding for growth

1.3 Stakeholders

Stake holder	Role	Interest
Project Owner / Manager	System Administrator	Creates projects, assigns tasks, monitors team
Team Member / Employee	End User	Receives assigned tasks, updates status, collaborates
Developer	Engineer	Builds, tests, and maintains the system
University As sessor	Evaluator	Reviews documentation and implementation quality

1.4 Problem Statement

Modern teams — in software development, project management, and academia — face persistent challenges coordinating work, tracking progress, and maintaining accountability. Traditional approaches (shared spreadsheets, email chains, physical boards) lack real-time synchronization, role differentiation, and data integrity.

No Centralized Visibility	Members and managers operate in silos with no single source of truth.
No Role-Based Access	Everyone can view and potentially modify data outside their scope.
Inefficient Progress Tracking	No automated mechanism to detect overdue items or track completion rates.
Poor Accountability	Without assignment history, it is impossible to audit task ownership.
No Real-Time Analytics	Decision-making suffers from manually aggregated, stale statistics.

2

Software Requirements Specification (SRS)

2.1 Functional Requirements

2.1.1 Authentication Module

ID	Requirement	Priority
FR-01	Users shall register with full name, email, password, and optional role selection	High
FR-02	System shall support Google OAuth 2.0 for social login	High
FR-03	Registered users shall log in with email and password	High
FR-04	JWT access tokens shall be issued upon successful login (30-day expiry)	High
FR-05	Authenticated sessions shall expire after configured timeout	Medium
FR-06	Users shall be able to log out and clear their session token	High
FR-07	Users shall update their profile name, avatar URL, and department	Medium

2.1.2 Manager Features

ID	Requirement	Priority
FR-08	Manager shall view a team analytics dashboard with performance metrics	High
FR-09	Manager shall create tasks with title, description, priority, deadline, assignee, and project	High
FR-10	Manager shall assign tasks to specific employees	High
FR-11	Manager shall edit any existing task in the system	High
FR-12	Manager shall delete tasks with confirmation	High
FR-13	Manager shall create and manage projects (CRUD)	High
FR-14	Manager shall view and manage all team members and their performance	Medium
FR-15	Manager shall reassign overdue tasks to other employees	Medium
FR-16	Manager shall view all tasks across all projects with filtering	High

2.1.3 Employee Features

ID	Requirement	Priority
----	-------------	----------

FR-17	Employees shall view a personal analytics dashboard	High
FR-18	Employees shall view all tasks assigned to them on the My Tasks page	High
FR-19	Employees shall update task status (todo → in_progress → done → blocked)	High
FR-20	Employees shall add comments to tasks for collaboration	Medium
FR-21	Employees shall create private tasks visible only to themselves	Medium
FR-22	Employees shall NOT reassign tasks to other users	High

2.1.4 Task & Project Management

ID	Requirement	Priority
FR-23	Tasks shall have statuses: Todo, In Progress, Blocked, Done	High
FR-24	Tasks shall support priority levels: Low, Medium, High	High
FR-25	Tasks shall support deadline dates with automatic overdue flag detection	High
FR-26	Tasks shall support tagging for categorization	Low
FR-27	Tasks shall support file attachment references (URL-based)	Low
FR-28	Projects shall have member lists managed by the Manager	High
FR-29	Projects shall support color coding and unique join codes	Medium
FR-30	Advanced filtering shall support status, priority, project, assignee, date range	High

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API response time under 500ms for 95% of requests under normal load
NFR-02	Security	All passwords hashed with bcryptjs (salt rounds = 10)
NFR-03	Security	All API routes protected by JWT middleware except public auth endpoints
NFR-04	Security	Google OAuth tokens validated server-side before account creation
NFR-05	Scalability	System shall support 500+ concurrent users via horizontal scaling
NFR-06	Usability	UI fully responsive on desktop and tablet screens via TailwindCSS
NFR-07	Availability	System shall target 99.5% uptime in production via cloud platform
NFR-08	Maintainability	Code shall follow modular MVC architecture with clear separation
NFR-09	Data Integrity	Database indexes on all frequently queried fields
NFR-10	Performance	Pagination enforced (default 20 items/page) for all list endpoints

2.3 Use Cases & User Stories

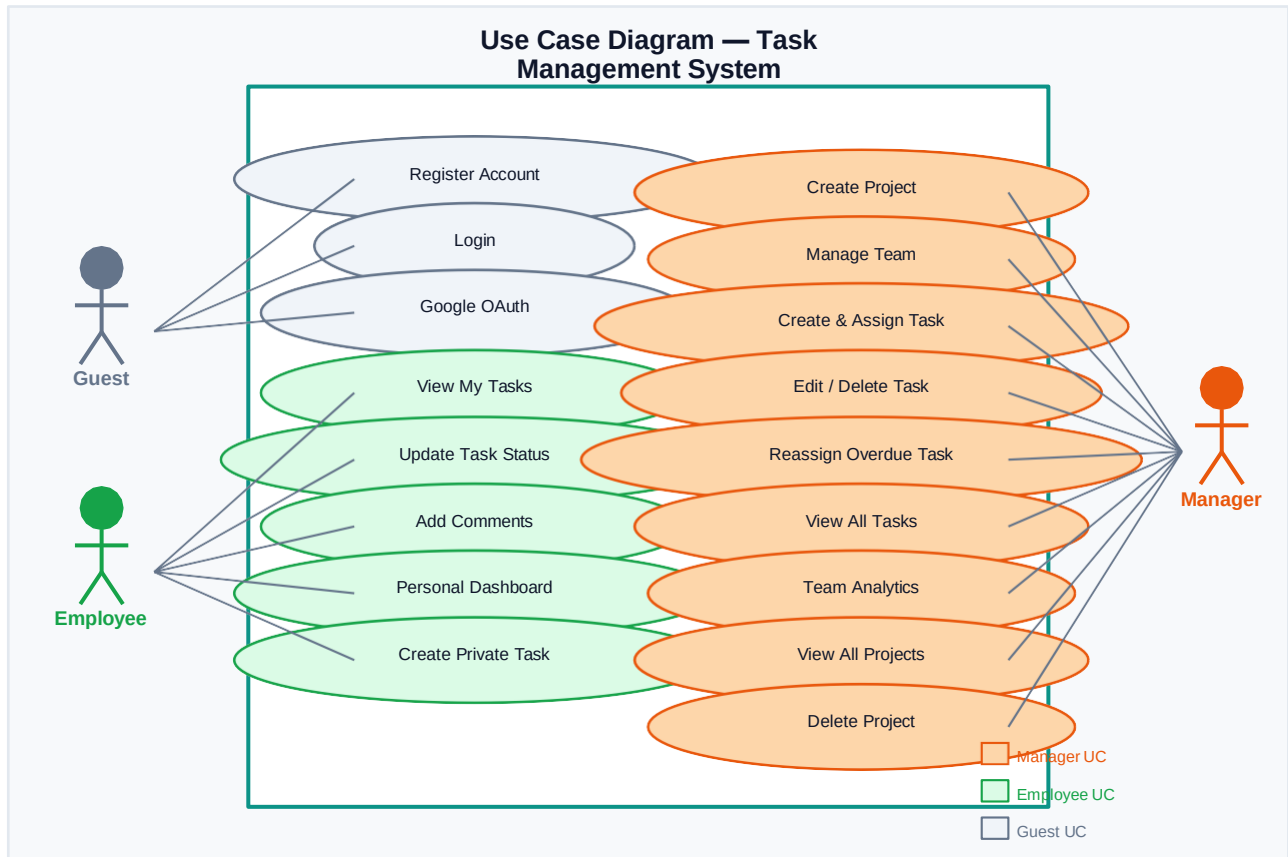


Figure 2.1 — Use Case Diagram — Task Management System

Actors:

- **Guest** — Unauthenticated visitor — can access signup/login only
- **Employee** — Authenticated team member with task-level access
- **Manager** — Authenticated administrator with full system control

User Stories (Agile Format):

ID	As a...	I want to...	So that...	Priority	Sprint
US-0 1	Guest	register with name, email, and password	I can access the system	High	1
US-0 2	Guest	log in with my credentials or Google OAuth	I have secure access	High	1
US-0 3	Manager	create tasks and assign them to employees	work is distributed properly	High	2
US-0 4	Manager	create and manage projects with team members	tasks are organized by context	High	2

US-0 5	Manager	edit and delete tasks	outdated information is corrected	High	2
US-0 6	Manager	view team analytics and performance	I can make data-driven decisions	High	3
US-0 7	Manager	reassign overdue tasks	deadlines are not missed	Medium	3
US-0 8	Employee	see only tasks assigned to me	my workload is clear	High	4
US-0 9	Employee	update task status from to do to done	progress is visible to the team	High	4
US-1 0	Employee	add comments to tasks	I can collaborate with colleagues	Medium	4
US-1 1	Employee	create private tasks	I can manage personal to-dos	Low	5
US-1 2	Employee	view personal analytics	I can track my own performance	Medium	5

2.4 System Constraints

Hardware	Development: Standard laptop (8GB RAM, Node.js compatible). Production: Cloud VPS or container.
Software	Node.js 14+, MongoDB 4.4+, React 18+, npm/yarn, Docker (optional for deployment)
Database	MongoDB Atlas M0 (free) for development; M10 cluster recommended for production
Third-party APIs	Google OAuth 2.0 (Google Cloud Console credentials required)
CORS	Backend must whitelist frontend origin; currently configured for localhost:3000
Token Storage	JWT stored in localStorage (v1.0); future version will migrate to HTTP-only cookies
File Uploads	v1.0 supports URL-based avatar references only; binary file uploads are a future enhancement

3

System Design

3.1 Architectural Design

TaskFlow follows a three-tier client-server architecture separating presentation, business logic, and data persistence. This Monolithic architecture was chosen over Microservices given the project scope — providing simplicity of deployment, shared data models, and easier debugging during development while remaining scalable via Docker.

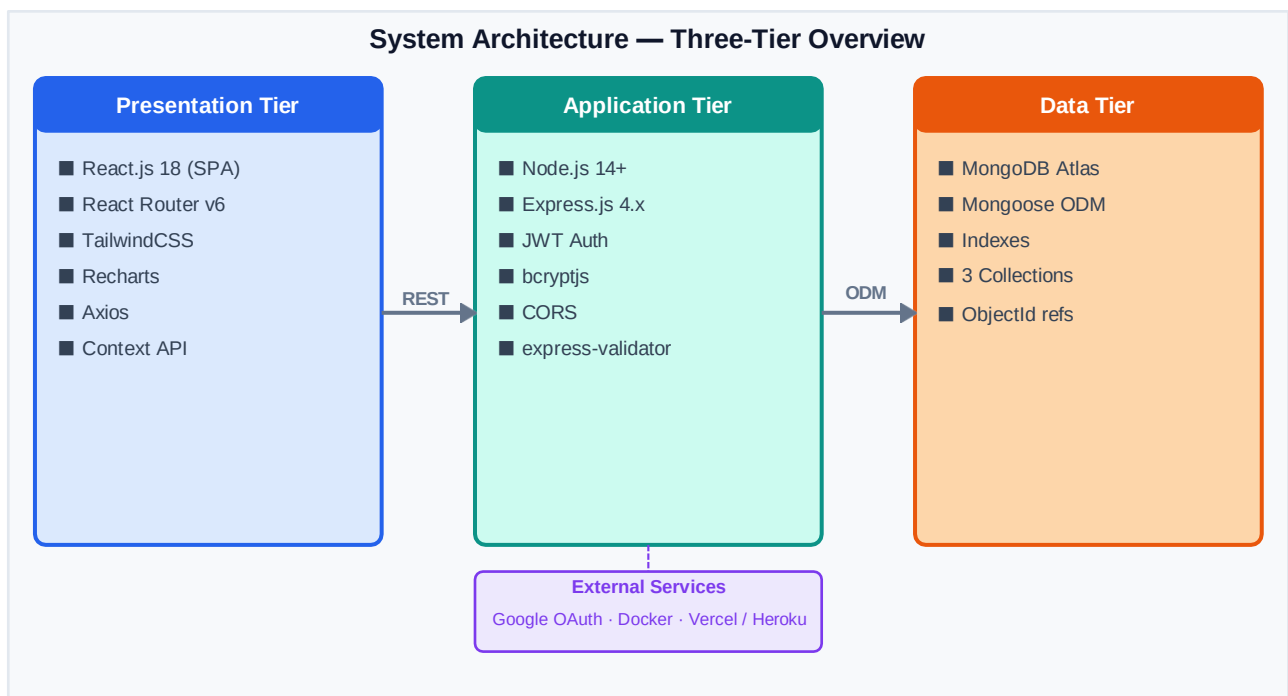


Figure 3.1 — System Architecture — Three-Tier Overview

Technology Stack:

Layer	Technology	Responsibility
Client (Frontend)	React.js 18 + Axios + Recharts	Render UI, manage state, call APIs
Routing	React Router v6	Client-side SPA navigation
Styling	TailwindCSS 3.x	Utility-first responsive design
State Management	React Context API	Global auth and user state
Notifications	React Hot Toast	Real-time feedback messages
API Server	Node.js + Express.js 4.x	Handle HTTP requests, business logic

Authentication	JWT + bcryptjs + Google OAuth	Token issuance, password hashing, social login
Database	MongoDB + Mongoose ODM	Persist all application data
Containerization	Docker + docker-compose	Portable, reproducible environments

3.2 Database Design

MongoDB is used as the primary datastore with Mongoose as the ODM. The database contains three primary collections: **users**, **tasks**, and **projects**. References between documents use ObjectId fields. Comments are embedded within task documents as an array (composition relationship) to avoid excessive joins.

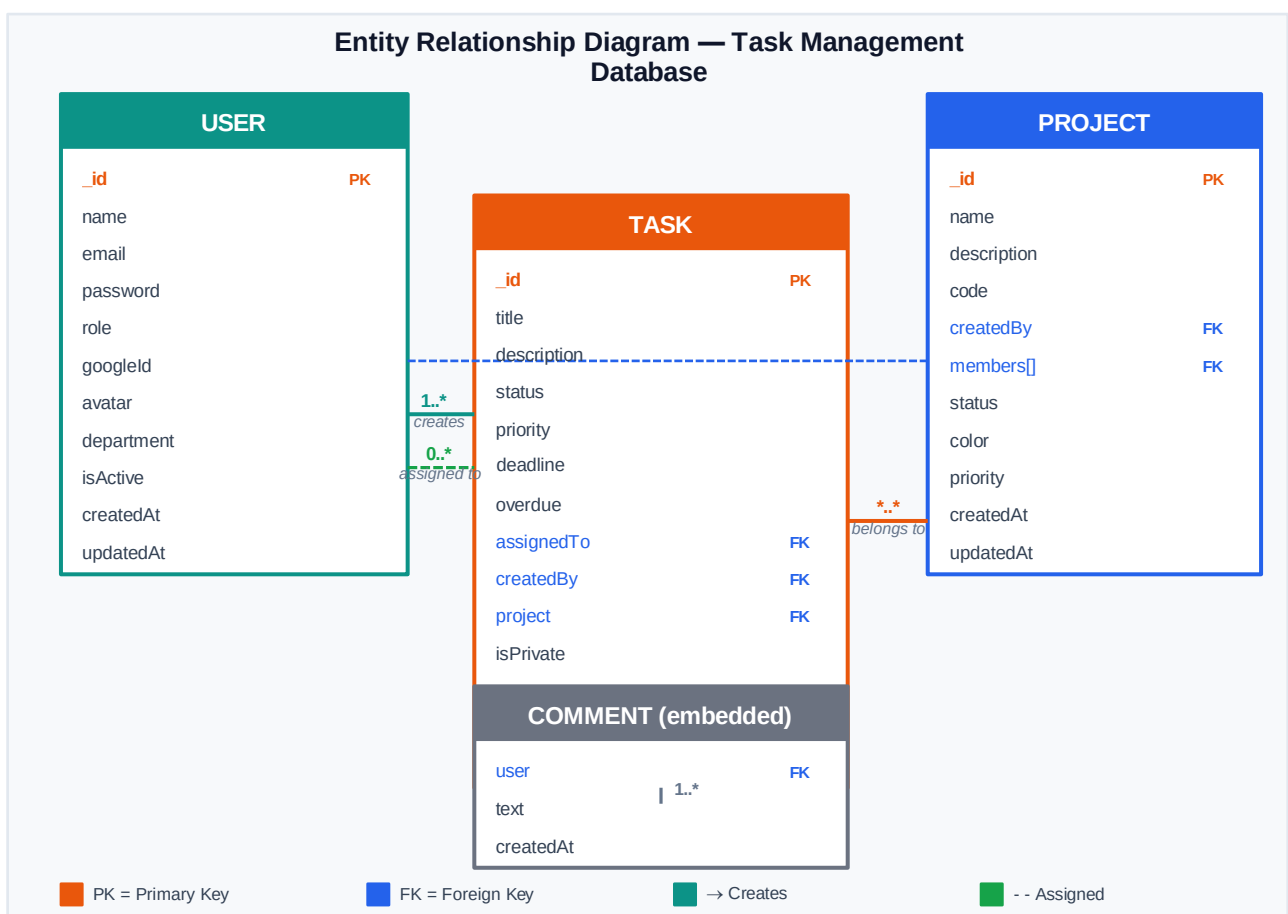


Figure 3.2 — Entity Relationship Diagram — TaskFlow Database

User Schema:

Field	Type	Constraints	Description
_id	ObjectId	Auto-generated	Unique document identifier
name	String	Required, maxLength:50	Full display name
email	String	Required, unique	Login identifier
password	String	minLength:6, hashed	bcryptjs hashed password
role	String	Enum: manager employee	Access level of account
google Id	String	Unique, sparse	Google OAuth identifier
avatar	String	Optional	Profile image URL
department	String	Optional	Team or department name
isActive	Boolean	Default: true	Account active status

Task Schema:

Field	Type	Constraints	Description
_id	ObjectId	Auto-generated	Unique task identifier
title	String	Required, maxLength:200	Short task title
description	String	maxLength:2000	Detailed description
status	String	Enum: todo in_progress blocked done	Task lifecycle state
priority	String	Enum: low medium high	Urgency level
deadline	Date	Optional	Target completion date
overdue	Boolean	Default: false	Auto-detected overdue flag
assignedTo	ObjectId	Ref: User	Assigned employee
createdBy	ObjectId	Ref: User, Required	Manager who created task
project	ObjectId	Ref: Project	Parent project
isPrivate	Boolean	Default: false	Private task flag
comments	[Object]	Array {user, text, date}	Embedded comments
tags	[String]	Array	Task tags for categorization
completedAt	Date	Optional	Completion timestamp

Project Schema:

Field	Type	Constraints	Description
_id	ObjectId	Auto-generated	Unique project identifier
name	String	Required, maxLength:100	Project display name
description	String	maxLength:500	Project description
code	String	Unique, auto generated	6-char join code
createdBy	ObjectId	Ref: User, Required	Manager who created project
members	[ObjectId]	Ref: User	Array of member IDs
status	String	Enum: active archived completed	Project lifecycle state
color	String	Default: #3B82F6	Project color for UI
priority	String	Enum: low medium high	Project urgency level

Indexing Strategy:

Collection	Field(s)	Type	Purpose
users	email	Unique	Fast login lookup and uniqueness guarantee
users	role	Single	Role-based queries for manager analytics
tasks	assignedTo	Single	Efficiently fetch tasks for a specific employee
tasks	status	Single	Dashboard filter and status aggregation
tasks	priority	Single	Priority-based filtering and sorting
tasks	deadline	Single	Sort and filter overdue tasks
tasks	project	Single	Fetch all tasks within a project
tasks	createdBy	Single	Fetch all tasks created by a manager
tasks	overdue	Single	Quick overdue task queries
projects	createdBy	Single	Manager's project list
projects	members	Single	Projects a user belongs to
projects	code	Unique	Fast join-by-code lookup

3.3 System Components

- bcryptjs password hashing with 10 salt rounds
 - Indexed fields for all frequently queried properties
 - Protected routes
- Role assignment
 - Pagination
 - AuthContext for global authentication state management

- Consistent response envelope: { success, data?, error? }

- express-asy

API Endpoints Reference:

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register a new user account
POST	/api/auth/login	Public	Login and receive JWT token
POST	/api/auth/google	Public	Google OAuth login
GET	/api/auth/me	User	Retrieve current user profile
PUT	/api/auth/update	User	Update name, avatar, department
GET	/api/tasks	User	List all tasks (with filters + pagination)
GET	/api/tasks/:id	User	Get single task details
POST	/api/tasks	User	Create a new task
PUT	/api/tasks/:id	User	Update task details
DELETE	/api/tasks/:id	User	Delete a task
POST	/api/tasks/:id/comments	User	Add comment to task
GET	/api/projects	User	List all projects (with pagination)
GET	/api/projects/:id	User	Get project details
POST	/api/projects	Manager	Create a new project
PUT	/api/projects/:id	Manager	Update project details
DELETE	/api/projects/:id	Manager	Delete a project
GET	/api/analytics/personal	User	Personal analytics data
GET	/api/analytics/manager	Manager	Team analytics dashboard data
GET	/api/analytics/users	Manager	List all users with pagination

3.4 UML Diagrams

Class Diagram

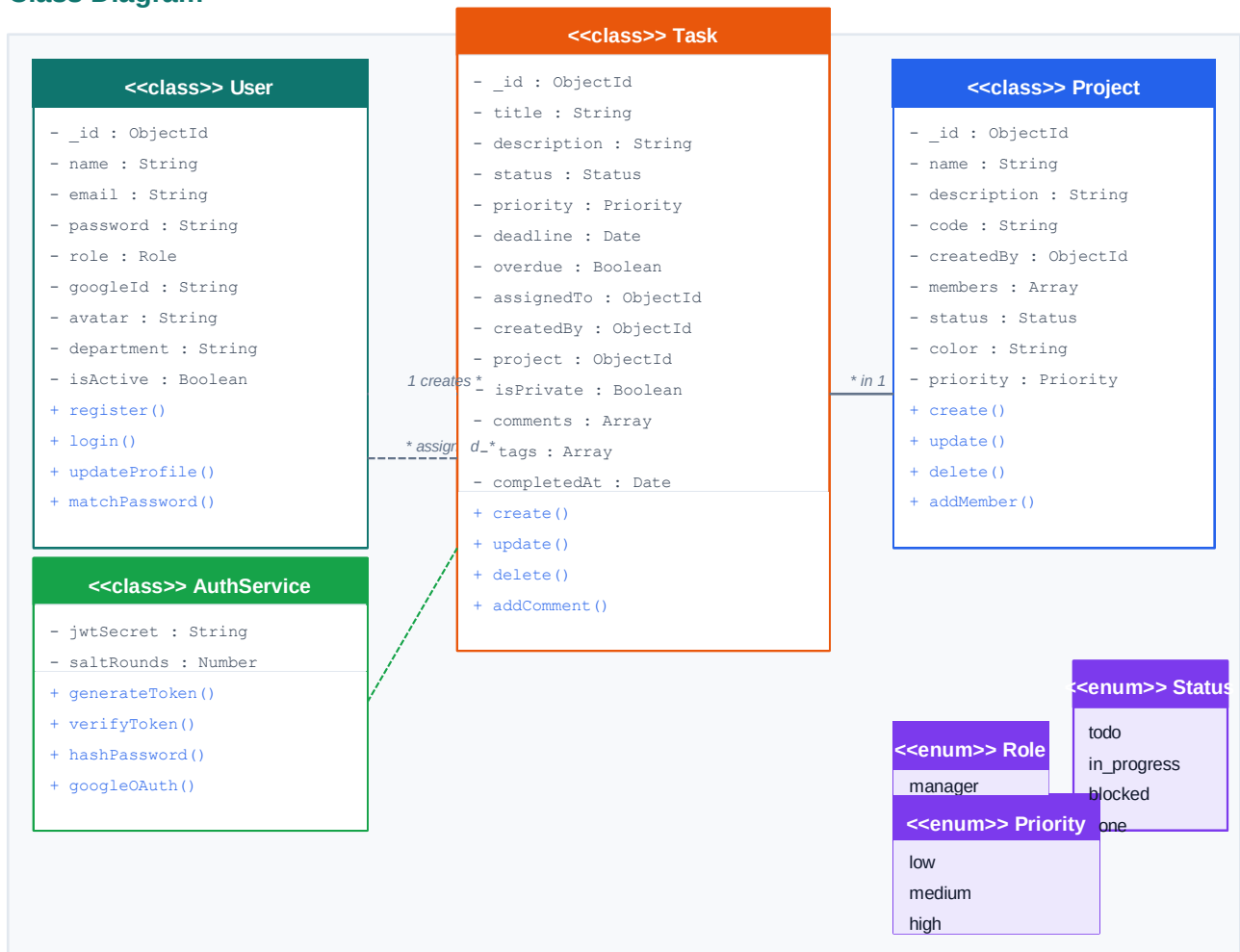


Figure 3.3 — Class Diagram — TaskFlow System

Core Classes:

Class	Key Attributes	Key Methods
User	<code>_id</code> , <code>name</code> , <code>email</code> , <code>password</code> , <code>role</code> , <code>googleId</code> , <code>avatar</code> , <code>department</code> , <code>isActive</code>	<code>register()</code> , <code>login()</code> , <code>updateProfile()</code> , <code>matchPassword()</code>
Task	<code>_id</code> , <code>title</code> , <code>description</code> , <code>status</code> , <code>priority</code> , <code>deadline</code> , <code>overdue</code> , <code>assignedTo</code> , <code>createdBy</code> , <code>project</code> , <code>isPrivate</code> , <code>comments[]</code> , <code>tags[]</code>	<code>create()</code> , <code>update()</code> , <code>delete()</code> , <code>addComment()</code>
Project	<code>_id</code> , <code>name</code> , <code>description</code> , <code>code</code> , <code>createdBy</code> , <code>members[]</code> , <code>status</code> , <code>color</code> , <code>priority</code>	<code>create()</code> , <code>update()</code> , <code>delete()</code> , <code>addMember()</code>
AuthService	<code>jwtSecret</code> , <code>saltRounds</code>	<code>generateToken()</code> , <code>verifyToken()</code> , <code>hashPassword()</code> , <code>googleOAuth()</code>

Sequence Diagram: User Login Flow

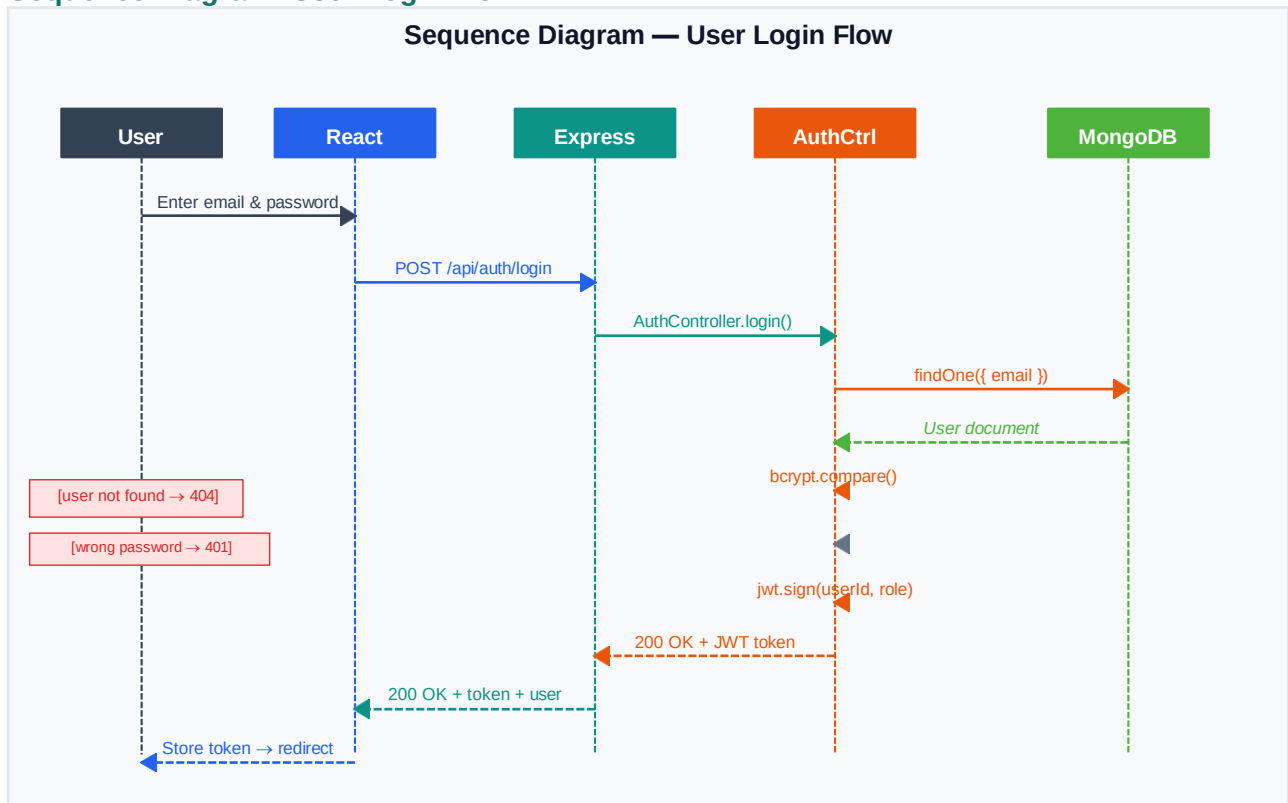


Figure 3.4 — Sequence Diagram — User Login Flow

Sequence Diagram: Create Task Flow

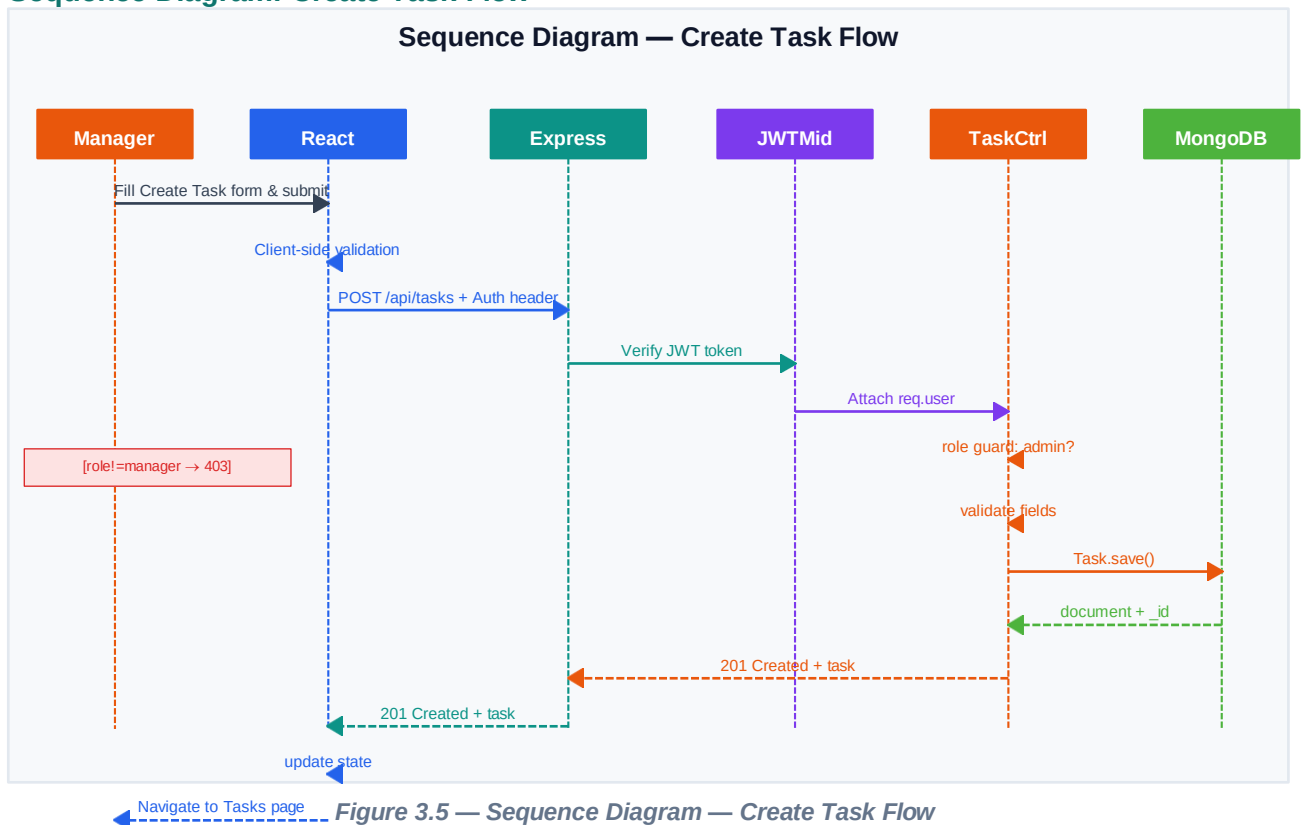


Figure 3.5 — Sequence Diagram — Create Task Flow

State Diagram: Task Lifecycle

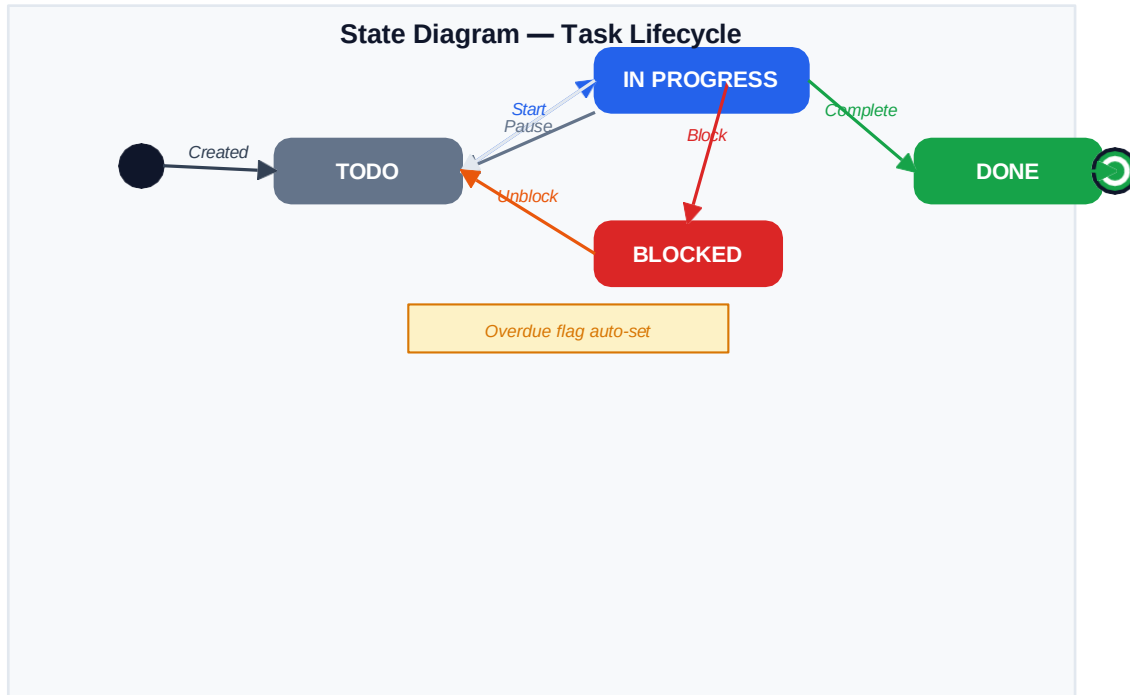


Figure 3.6 — State Diagram — Task Lifecycle Management

Tasks transition between four states: **Todo** (initial), **In Progress** (active work), **Blocked** (impediment exists), and **Done** (completed). The **overdue** flag is automatically set when the deadline passes without reaching Done status. Managers can reassign overdue tasks; employees can update status but cannot change task ownership.

4

Implementation

4.1 Development Plan

The project follows a Scrum-inspired Agile methodology with two-week sprints. Each sprint produces a working, tested increment. A product backlog is maintained and prioritized before each sprint planning session.

Sprint Plan:

Sprint	Duration	Goal	Key Deliverables
Sprint 0	Week 1	Project Setup	Repo init, folder structure, MongoDB Atlas, Docker config, CI skeleton
Sprint 1	Weeks 2–3	Authentication Module	User model, register/login APIs, JWT middleware, Google OAuth, React auth pages
Sprint 2	Weeks 4–5	Task Core CRUD	Task model, CRUD APIs, role guards, task list and detail pages
Sprint 3	Weeks 6–7	Project Management	Project model, CRUD APIs, project pages, member management
Sprint 4	Weeks 8–9	Dashboard & Analytics	Analytics API, KPI cards, Recharts bar/pie charts, team metrics
Sprint 5	Weeks 10–11	Filtering & Comments	Advanced filter system, comment feature, private tasks, overdue detection
Sprint 6	Weeks 12–13	Testing & QA	Unit tests, integration tests, E2E testing, bug fixes, accessibility audit
Sprint 7	Week 14	Deployment & Polish	Docker production build, cloud deploy, performance optimization, documentation

Product Backlog (Priority Order):

ID	User Story	Points	Sprint
US-01	As a guest, I want to register so I can access the system	5	Sprint 1
US-02	As a guest, I want to log in with credentials or Google OAuth	5	Sprint 1
US-03	As a manager, I want to create tasks and assign to employees	8	Sprint 2
US-04	As a manager, I want to create and manage projects	8	Sprint 3
US-05	As a manager, I want to edit and delete tasks	5	Sprint 2

US-06	As a manager, I want to view team analytics and performance	8	Sprint 4
US-07	As a manager, I want to reassign overdue tasks	5	Sprint 5
US-08	As an employee, I want to see only tasks assigned to me	5	Sprint 2
US-09	As an employee, I want to update task status	5	Sprint 2
US-10	As an employee, I want to add comments to tasks	3	Sprint 5
US-11	As an employee, I want to view personal analytics	5	Sprint 4
US-12	As a user, I want to filter tasks by multiple criteria	8	Sprint 5

Version Control Strategy (Git/GitHub):

- **main** branch — production-ready code only; protected with required PR reviews
- **develop** branch — integration branch for merging completed features
- **feature/** branches — one branch per user story (e.g., feature/task-filtering)
- **fix/** branches — isolated bug fixes before merging to develop
- Commit messages follow conventional commits: feat:, fix:, docs:, test:
- Pull Requests require at least one review approval before merge

4.2 Coding Best Practices

Clean Code Principles

- Meaningful, descriptive variable and function names throughout codebase
- Single Responsibility Principle — each function/component handles one concern
- DRY (Don't Repeat Yourself) — reusable utility functions in /utils/helpers.js
- ES6+ syntax: arrow functions, destructuring, async/await, template literals
- Functional React components with hooks — no class components

Modularization & Reusability

- MVC architecture: Models → Controllers → Routes separation on backend
- Common UI components in /components/common/ (Button, Card, Input, Modal, TaskCard)
- Centralized API service layer in /services/api.js using Axios instances
- Custom hooks (useAuth) encapsulate authentication logic across components
- Context API for global state — avoids prop-drilling in nested components

Environment & Security

- All secrets in .env files; .env.example provided for onboarding
- JWT_SECRET, MONGODB_URI, GOOGLE_CLIENT_ID never committed to version control
- .gitignore excludes node_modules, .env, build artifacts

- CORS configured to whitelist specific frontend origin only
- SQL injection prevented by Mongoose schema validation and ODM layer
- XSS protection via React's default JSX escaping; server validates all inputs
- CSRF mitigation via SameSite cookie attributes (future: HTTP-only cookies)
- Rate limiting recommended for auth endpoints in production (express-rate-limit)

4.3 Testing & Debugging

The testing strategy employs a layered approach covering unit, integration, and end-to-end testing. The goal is to achieve $\geq 80\%$ code coverage across critical business logic while ensuring all user journeys function correctly end-to-end.

Level	Tool	Scope	Coverage
Unit Testing	Jest + React Testing Library	Controllers, services, utility functions, React components	□ 80%
Integration Testing	Supertest + Jest	API endpoints with in-memory MongoDB (jest-mongodb)	□ 70%
E2E Testing	Cypress	Critical flows: register, login, create task, filter, analytics	Key flows 100%
Manual Testing	Postman + Browser	API smoke tests, visual UI validation, cross-browser check	All features
UAT	Stakeholder review	Real scenarios with Manager and Employee roles on staging environment	All stories

Key Test Cases:

Test ID	Description	Expected Result
TC-01	Submit valid registration form	201 Created, user in DB, JWT returned
TC-02	Register with duplicate email	400 Bad Request — email already in use
TC-03	Login with correct credentials	200 OK, JWT token returned
TC-04	Login with wrong password	401 Unauthorized
TC-05	Access protected route without token	401 Unauthorized
TC-06	Employee accesses Manager-only route	403 Forbidden
TC-07	Manager creates task with all fields	201 Created, task persisted
TC-08	Create task without required title	400 Bad Request with validation error
TC-09	Manager updates task status	200 OK, status field updated
TC-10	Manager deletes a task	200 OK, task removed from DB
TC-11	Employee adds comment to task	200 OK, comment embedded in task
TC-12	Employee tries to create a project	403 Forbidden
TC-13	Filter tasks by status=done and priority=high	200 OK, filtered results
TC-14	Google OAuth with valid Google token	200 OK, user created/found, JWT returned

5

Deployment & Evolution

5.1 Deployment Strategy

Environments:

Environment	Purpose	Infrastructure
Development	Local coding and unit testing	localhost:3000 (React) + localhost:5000 (Express)
Staging	Integration testing and QA sign-off	Heroku review app / Vercel preview deployment
Production	Live user-facing system	Heroku (API) + Vercel (Frontend) + MongoDB Atlas

Deployment Architecture:

- **Frontend (React SPA)** deployed to Vercel with automatic builds from the main branch
- **Backend (Node/Express API)** deployed to Heroku using the included Dockerfile
- **Database:** MongoDB Atlas M0 (free) for staging; M10 cluster for production data
- **Docker Compose** available for local and self-hosted deployment of both services
- **Environment variables** managed via Heroku Config Vars and GitHub Secrets

CI/CD Pipeline (GitHub Actions):

1. Developer pushes code to a feature branch
2. Pull Request triggers the CI workflow automatically
3. Workflow: install dependencies → ESLint lint → run all Jest tests
4. If all checks pass, PR may be merged to main branch
5. Merge to main triggers automatic deployment to Heroku (API) and Vercel (Frontend)
6. Deployment health check pings /api/health endpoint post-deploy
7. Slack / email notification on deployment success or failure

Environment Variables:

Variable	Location	Purpose
MON_GODDB_URI	Backend	MongoDB Atlas connection string
JWT_SECRET	Backend	Secret key for signing JWT tokens
GOOGLE_CLIENT_ID	Backend + Frontend	Google OAuth app client ID
GOOGLE_CLIENT_SECRET	Backend	Google OAuth app client secret
CLIENT_URL	Backend	CORS allowed origin (frontend URL)

NODE_ENV	Backend	Environment flag (development / production)
PORT	Backend	Server port (default 5000)
REACT_APP_API_URL	Frontend	Base URL of the deployed backend API

Production Checklist:

- HTTPS enforced on all endpoints (Vercel and Heroku provide automatic TLS)
- CORS configured to allow only the production frontend origin

- JWT_SECRET rotated from development value to a cryptographically strong secret
- MongoDB Atlas IP allowlist restricted to Heroku outbound IPs
- Error logging enabled; sensitive data (passwords, tokens) excluded from logs
- Database connection pool size tuned for expected concurrent load
- All .env files excluded from version control; secrets in hosting dashboards
- Google OAuth redirect URIs updated to production domain in Google Cloud Console

5.2 System Monitoring

Logging & Monitoring Tools:

Tool	Purpose
Morgan	HTTP request logger middleware — logs method, URL, status, response time in production
Winston	Structured JSON application logging — error and info levels with file transport
Heroku Metrics	Built-in dashboard for response times, memory usage, and dyno restarts
MongoDB Atlas	Built-in monitoring for query performance, index hit ratios, and slow queries
UptimeRobot	Free external uptime monitoring — pings /api/health every 5 minutes
Sentry (planned)	Error tracking and alerting for uncaught backend exceptions in production

Performance Metrics Tracked:

API Response Time	Target < 500ms for 95th percentile; monitored via Morgan logs
Error Rate	HTTP 4xx/5xx rates tracked; alert threshold > 1% of requests
Uptime	Target 99.5% monthly; monitored externally via UptimeRobot
Database Query Time	Slow query log enabled for queries > 100ms in MongoDB Atlas
Memory Usage	Heroku dyno memory monitored; restart triggered on leak detection
Task Completion Rate	Business metric tracked in analytics; signals user engagement health

5.3 Maintenance & Evolution

Bug Fix & Security Patch Strategy:

- Critical security vulnerabilities patched within 24 hours via hotfix branches
- P1 bugs (system unavailable) fixed within 48 hours; P2 (major feature broken) within 1 week
- npm audit run weekly in CI pipeline; Dependabot PRs reviewed monthly
- Security advisories from Node.js, MongoDB, and key npm packages monitored continuously

Feature Update & User Feedback Plan:

- Bi-weekly sprint reviews with stakeholders to gather prioritized feedback
- In-app feedback mechanism planned for v1.1 (embedded form or Intercom widget)
- Feature requests tracked in GitHub Issues with enhancement label
- A/B testing for significant UI changes before full rollout

Scalability Roadmap:

v1.1 — Short Term

- WebSocket integration (Socket.io) for real-time task update notifications
- File upload and attachment management via AWS S3 or Cloudinary
- Email notifications for task assignments, deadlines, and comments
- HTTP-only cookie migration for JWT to improve XSS resistance

v1.2 — Medium Term

- Redis caching layer for analytics queries and session management
- Task templates and repeating/recurring task automation
- Bulk task operations (multi-select, bulk status update)
- CSV/PDF export for task reports and analytics data
- Slack and calendar integrations (Google Calendar, Outlook)

v2.0 — Long Term

- React Native mobile application with offline support and push notifications
- Single Sign-On (SSO) via SAML 2.0 for enterprise customers
- Advanced role system with granular permissions per project
- Microservices extraction for analytics and notification services
- Load balancing with NGINX and horizontal scaling via Kubernetes
- Audit log system for compliance and change history tracking